

Keywords: tabular foundation models • FP8 quantization • in-context learning • attention speedup • TabPFN

Attention Quantization for Tabular Foundation Models

Tabular FM KV Matrices Scale Quadratically With Training Rows — FP8 Quantization With Test-Training Row Alignment Achieves 1.7× Speedup at Zero Accuracy Cost

By Jonas M. Kübler, Benjamin Jäger, Klemens Flöge, Noah Hollmann, Frank Hutter (University of Freiburg; ELLIS Institute Tübingen)

arXiv:2609.13031

Tabular foundation models such as TabPFN and TabICL perform in-context learning on new datasets by encoding the entire training set as a sequence of tokens and attending over all rows in a single forward pass. Their attention matrices scale quadratically with training set size — a table with 10,000 rows and 4 attention heads contains 400 million attention entries per layer, dwarfing the KV caches of generative LLMs. FP8 quantization has transformed LLM serving, but applying it to tabular FMs requires solving a unique alignment problem: the quantization error on test rows must match that on training rows, or prediction accuracy collapses.

AT A GLANCE

Problem: Tabular FM KV matrices scale $O(n^2)$ with training rows; FP8 quantization without row-group alignment degrades accuracy by 3–5 points.

Why it matters: At $n = 10,000$ training rows, FP16 attention costs 800 MB per layer; FP8 halves this and doubles bandwidth.

Method: Dynamic per-group FP8 scale calibration with test/train row alignment; custom asymmetric-tile Triton kernel.

Result: 1.7× speedup on H100 with <0.2-point accuracy change on TabPFN-v3 and TabICLv2.

Evidence: TabArena (47 datasets), BeyondArena (21 datasets); ablations on alignment, format, tile shape.

The Big Picture

In a tabular FM, all n training rows and m test rows are encoded as tokens and processed in one forward pass. The attention matrix $A \in \mathbb{R}^{(n+m) \times (n+m)}$ has $(n+m)^2$ entries per head per layer. For $n = 10,000$, $m = 1,000$, and 4 heads at FP16:

$$\text{memory} = 4 \times (1.1 \times 10^4)^2 \times 2 \text{ B} \approx 968 \text{ MB per layer}$$

This exceeds the KV cache of a 7B LLM processing a 128k-token context at the same precision. FP8 quantization halves this cost and doubles effective bandwidth utilization.

The Quantization Error Misalignment Problem

Standard FP8 quantization assigns a single scale s to a tensor X :

$$s = \frac{\|X\|_\infty}{448}, \quad \hat{X} = \text{round}(X/s) \cdot s$$

(448 is the maximum value in FP8 e4m3fn format.)

In a tabular FM, the query matrix $Q = [Q_{\text{train}}; Q_{\text{test}}]$ has two row groups with different distributions: training rows include label information in their embeddings; test rows do not. If $\|Q_{\text{train}}\|_\infty \gg \|Q_{\text{test}}\|_\infty$, then s is calibrated to training rows, and test queries use only the low end of the FP8 range — far fewer distinct values — introducing systematic bias in test attention scores.

Core Mechanism: Per-Group Dynamic Alignment

SAS applies separate dynamic scales per row group:

$$s_g = \frac{\|Q_g\|_\infty}{448}, \quad g \in \{\text{train}, \text{test}\}$$

Computed from the tensor itself at each forward pass (no calibration dataset). This equalizes relative quantization error:

$$\frac{\epsilon_q}{\|Q_g\|_\infty} \leq \frac{1}{2 \times 448} \approx 0.001$$

for both row groups, eliminating the systematic bias.

Technical Formulation

FP8 e4m3fn: 4 exponent bits, 3 mantissa bits, 1 sign bit. Representable range $[-448, 448]$; step size $s/8$ per scale.

Per-group quantized attention:

$$\hat{Q}_g = \text{round}(Q_g/s_g) \cdot s_g$$

$$A_{ij} = \text{softmax}\left(\frac{\hat{\mathbf{q}}_i \hat{\mathbf{k}}_j^\top}{\sqrt{d}}\right), \quad \mathbf{O}_i = \sum_j A_{ij} \hat{\mathbf{v}}_j$$

Alignment overhead: One pass over the input to compute per-group max values; <1% of total attention compute time.

Speedup bound: Bandwidth-bound regime (large n):

$$\text{Speedup}(n) \approx \frac{16}{8} \cdot \frac{B_{\text{FP16}}}{B_{\text{FP8}}} = 2 \times (1 - \epsilon_{\text{kernel}})$$

with kernel overhead $\epsilon_{\text{kernel}} \approx 15\%$ for the custom Triton kernel.

Experimental Results

Tested on TabPFN-v3 and TabICLv2 on an H100 GPU:

Model	Precision	Speedup	Acc. Change
TabPFN-v3	FP16	1.00×	—
TabPFN-v3	FP8 (no align)	1.65×	−3.1%
TabPFN-v3	FP8 (aligned)	1.70×	−0.1%
TabICLv2	FP16	1.00×	—
TabICLv2	FP8 (no align)	1.62×	−4.7%
TabICLv2	FP8 (aligned)	1.68×	−0.2%

Alignment eliminates 3–5 point accuracy degradation; the residual <0.2 points is within noise. Benchmarks: TabArena (47 classification datasets), BeyondArena (21 regression datasets).

Ablations and Interpretation

Without per-group scale (global scale): Accuracy drops 3.1–4.7 points — the distribution mismatch between test and training rows is the dominant failure mode.

E4m3fn vs. E5m2: E4m3fn (more mantissa bits) outperforms E5m2 (more exponent bits) because attention logit distributions are well-concentrated and benefit more from mantissa precision.

Tile shape: Custom asymmetric tiling adds 18% speedup over reusing a standard square-tile FP8 kernel, justified by the asymmetric $m \times n$ ($m \ll n$) shape of tabular FM attention.

Scaling with n : Speedup grows from $1.3\times$ at $n = 500$ to $1.9\times$ at $n = 10,000$, reflecting that larger attention matrices are more bandwidth-bound.

Reference

Jonas M. Kübler, Benjamin Jäger, Klemens Flöge, Noah Hollmann, Frank Hutter. **Attention Quantization for Tabular Foundation Models.** arXiv:2609.13031, September 2026. <https://arxiv.org/abs/2609.13031>